

# Deep Tuning Reinforcement Learning applied to chess

<https://github.com/IAntoineV/ChessRL>

CentraleSupélec

Antonine Batifol

Kerrian Le Caillec

Antoine Vialle

## Abstract

*Reinforcement learning (RL) has shown success in decision-making tasks, with chess serving as a fundamental testbed for AI advancements. This report investigates the use of Group Relative Policy Optimization (GRPO) for training a supervised policy network in a chess-playing agent. Initially, a neural network is pre-trained on expert game data to learn a strong policy through supervised learning. The policy is then iteratively refined using fine-tuning methods, which optimizes decision-making by balancing policy improvement and stability. Our experiments show that this approach accelerates learning, enhances policy performance, and effectively integrates supervised learning with reinforcement learning. These findings highlight the potential of GRPO in strategic environments like chess, where leveraging both human knowledge and self-improvement is crucial for optimal play.*

## 1. Introduction and Motivation

Reinforcement learning has been widely used as a tool to learn from opponent-based environments such as board games. Chess, which is a classical domain for RL research, provides an ideal testbed due to its vast availability of data notably games from highly rated players. The success of deep RL models such as AlphaZero demonstrated the power of this model in mastering chess, shogi, and Go through self-play, surpassing human expertise using deep neural networks [3, 4]. More recently, similar techniques have been adapted for fine-tuning large language models (LLMs), such as in DeepSeek’s thinking mode, which was initially made for mathematical reasoning, in which reinforcement-based strategies enhance reasoning abilities. However, many state-of-the-art deep tuning methods demand substantial computational resources, making them inaccessible to researchers without high-end hardware. Our motivation is to explore less expensive deep tuning methodologies that leverage traditional RL principles.

## 2. Deep learning agents

In this section we will discuss about the agents we modelled using deep learning architectures for the policy network. We will first introduce the quantitative aspect of the game environment and how we can encode the useful information about the state of the game for the policy network.

### 2.1. Definition of the states

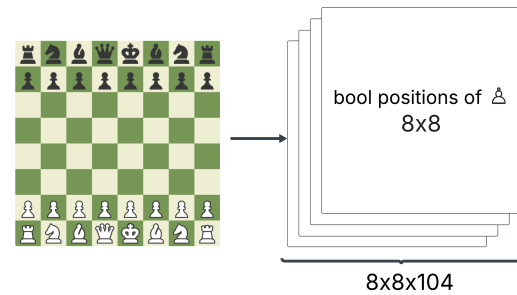


Figure 1. Model state

We derive our state representation (Fig. 1) from the board’s FEN (Forsyth-Edwards Notation) encoding and convert it to a tensor of dimension  $(8 \times 8 \times 104)$ . This tensor encodes the board state (which is of size  $8 \times 8$ ) and:

- For the current board, the placement of the pieces makes for 6 types of piece and 2 color (12 planes),
- 7 boards history ( $7 \times 12$  planes),
- whose turn is it to play 1 (1 plane),
- Castling rights in chess (4 plane),
- The “50-move” draw rule of chess (1 plane),
- the ELO rating of the player (1 plane),
- The Time Control of the game which is how much time each player receives to complete a game (1 plane), which represents a total of 104 planes.

### 2.2. Definition of the actions space

The action space consists of 1858 possible moves. We obtain the actions space by displacing the above mask on each case of the chess board. The mask encode the possible type of displacement of all existing piece. To do so, the mask is

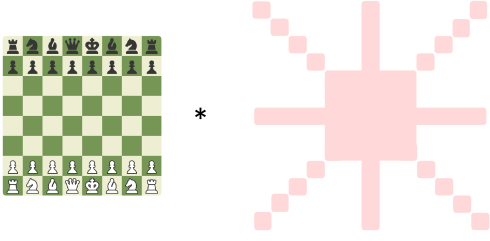


Figure 2. Action space

convoluted with the  $8 \times 8$  chess board. We also need to add all possible promotions to the action space. In Fig. 2, we have describe the binary mask as it is in the code. We have considered every type of possible displacements into this binary mask. Another step to ensure the legality of the policy outputted by the agent will be necessary as these moves will need to be piece-dependent.

### 2.3. Definition of the model architecture

### 3. Agent Training

The training process for the agent consists of two main stages: supervised pretraining, and tuning with either Group Relative Policy Optimization (GRPO) or distillation tuning.

#### 3.1. Pretrained Model

Initially, the model is trained in a supervised learning setup using a dataset of games played by high-level human players with ELO ratings greater than or equal to 2200. This ensures the model is exposed to strong, strategic gameplay patterns. The model architecture is based on the Lee-laZero framework [1, 3]. The overall architecture of the deep model is given in Fig. 3, we repeat several times a transformer architecture and a specific attention mechanism for the model to better understand the interactions in the board. During each supervised iteration, a tuple of board state and the human-played move  $(s, a)$  is sampled. The model produces a predicted action distribution  $\pi(s)$ , and the training loss is computed using the cross-entropy (CE) loss between the predicted policy and the ground truth move:  $\mathcal{L} = CE(\pi(s), a)$ .

#### 3.2. GRPO Tuning

After the supervised stage, the model can be fine-tuned using Group Relative Policy Optimization (GRPO) [2], which refines the policy through interaction with the Stockfish chess engine. In each GRPO iteration, the current policy  $\pi_\theta$  updates the reference policy  $\pi_{\text{ref}}$ . Outputs  $\{o_i\}_i$  are sampled from  $\pi_{\text{ref}}(s)$  and evaluated by Stockfish to obtain rewards  $r_i = r_\phi(o_i)$  (here the parameters  $\phi$  are fixed by the engine and we cannot change the parameter values). Advantages are computed as normalized rewards:  $\hat{A}_i = \frac{r_i - \text{mean}(r_i)}{\text{std}(r_i)}$ .

The probability ratio between the current and reference policy is calculated as:

$$r(\theta) = \exp(\log \pi_\theta(a|s) - \log \pi_{\text{ref}}(a|s)).$$

Using this, the surrogate loss is defined as:

$$L_s = \min \left( r \cdot \hat{A}, \text{clip}(r, 1 - \epsilon, 1 + \epsilon) \cdot \hat{A} \right),$$

which stabilizes updates similarly to PPO. Additionally, the KL divergence between the current and reference policy is computed:

$$D_{KL}(\pi_\theta || \pi_{\text{ref}}) = \sum_i \pi_{\theta,i} \log \left( \frac{\pi_{\theta,i}}{\pi_{\text{ref},i}} \right).$$

The final objective is:

$$\mathcal{L}_{\text{GRPO}} = \text{clip}(-\mathbb{E}[L_s] + D_{KL}, -1, 1),$$

which balances reward maximization with regularization to ensure policy updates remain within a trust region. A distinctive feature of GRPO compared to traditional PPO is the use of a grouped reference policy obtained by sampling multiple outputs from the current policy’s softmax distribution, rather than requiring a separate value network.

#### 3.3. Distillation Tuning

After the supervised stage, the model can be fine-tuned using distillation tuning to further align its policy with Stockfish’s evaluations. For each iteration, the reference policy updates the reference policy. Multiple outputs are sampled from  $\pi_{\text{ref}}(s)$  and evaluated by Stockfish to yield rewards  $r_i = r_\phi(o_i)$ . The best move, as determined by Stockfish, is represented as a one-hot policy  $\pi_{\text{best}}$ . The loss is computed using cross-entropy between the model’s predicted distribution and the Stockfish-best move:

$$\mathcal{L}_{\text{distill}} = CE(\pi, \pi_{\text{best}}).$$

This stage helps refine the policy by directly minimizing divergence from the strongest evaluated actions in the set of reference considered moves.

## 4. Classical RL chess agents

### 4.1. Monte-Carlo Tree Search

Monte-Carlo Tree Search can be a powerful algorithm for games like chess, where the search space is vast and exhaustive search is impractical. Unlike methods such as alpha-beta pruning, which systematically explore the game tree to a certain depth, MCTS focuses on analyzing the most promising moves by expanding the search tree based on random sampling of the search space, which allowing

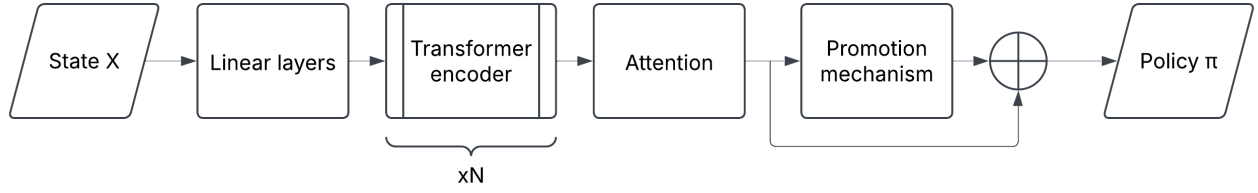


Figure 3. Overall architecture of the policy network

it to navigate vast search spaces and make decisions without exhaustive enumeration. MCTS builds its knowledge dynamically during gameplay by exploring a set number of iterations before selecting a move.

The MCTS Algorithm involve four stages at each iteration:

1. **Selection:** Starting from the root node, we recursively select the best child node until reaching a leaf. To select the child we use the UCB1 (Upper-confidence bound for tree):

$$child = \arg \max_{c \in children} V(c) + K \sqrt{\frac{\log(N)}{n(c)}}$$

where  $N$  refers to the number of visits of the parent,  $n(c)$  the number of visits of the child,  $K$  is the exploration parameter and  $V(c)$  the current value of the child. This formula allows a balance between exploration and exploitation.

2. **Expansion:** Once a leaf node is reached and if it is not a terminal state, we expand this node, by adding nodes from this leaf, that correspond to the new states generated by any legal move from the leaf state.
3. **Rollout/Simulation:** From the newly expanded node, the algorithm perform a simulation, either of the full game or for a specified depth. In this part, the algorithm can either select the next move randomly, which was the initial MCTS implementation, or select the move according to a policy, like in AlphaZero, using a neural network or an evaluation function, we can also use a mix with an  $\epsilon$ -greedy policy.
4. **Backpropagation:** Once the simulation has finished we backpropagate the results upwards to the root and update the value and increment the visit count of each node of this path.

In our application of MCTS represents the possible game states as tree, where:

- each node represents a game state: a chess board position
- each edge represents a legal move leading to the possible new state
- the root corresponds to the actual current chess board in the real game

Once the MCTS Algorithm has performed the max iteration number (or reach the time limit granted for the tree search), it returns either the root's child with the highest number of visits or the child with the highest value, which will become the next state of the real game.

## 4.2. Alpha-Beta Pruning

The Alpha-beta pruning algorithm enhances the minimax search tree algorithm, where the layers of the tree alternate between minimizer node and maximizer node. It applies branch-and-bound technique to prune unnecessary branches without overlooking a better move. To do so the algorithm maintains two value  $\alpha$  and  $\beta$ , where  $\alpha$  represents the minimum score the maximizing player is already assured to have at a step during the search and  $\beta$  the maximum score the minimizing player is assured.

The Alpha-Beta pruning algorithm (Fig.

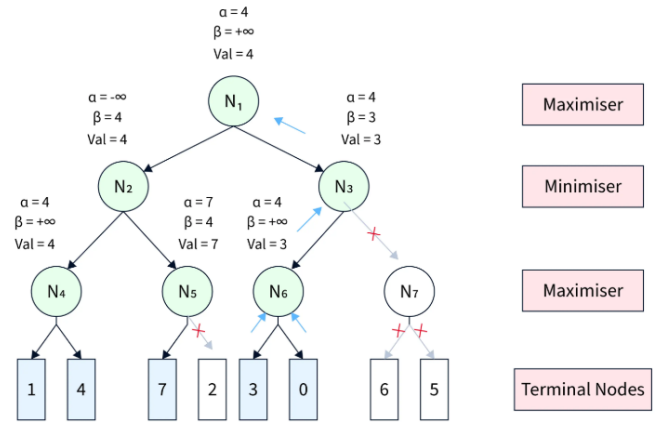


Figure 4. Alpha-Beta pruning algorithm

To improve efficiency, we apply move ordering, prioritizing nodes in decreasing value order. Evaluating strong moves first increases the likelihood of early cutoff and significantly reduce the number of nodes to search. This gain in efficiency allow for deeper search and thus potentially the detection of stronger move.

### 4.3. Custom reward to evaluate a chess board state

Monte Carlo Tree Search (MCTS) and Alpha-Beta Pruning rely on board evaluation to define a reward to propagate or to decide the pruning of branch. Given chess’s vast search space, full simulations and brute-force searches are inefficient, that’s why we specified the max depth to explore and need to evaluate the board before endgame. It is also useful to improve performance:

- **MCTS:** Instead of relying on pure playout results (random simulations), a heuristic evaluation *guides rollouts*, making the agent play more meaningfully.
- **Alpha-Beta:** The evaluation function *approximates deeper searches* allowing to search in a depth-limited tree. The evaluation is also useful to ordering the move.

We implemented a custom evaluation function based on Tomasz Michniewski’s Simplified Evaluation Function to assess board positions efficiently. It combines material value, positional bonuses, and endgame adaptation.

1. **Material evaluation:** Pieces have standard centipawn values (Pawn: 100, Knight: 320, Bishop: 330, Rook: 500, Queen: 900 and King: 20000).
2. **Positional evaluation:** We use piece-square tables which act as weighted masks, assigning different values to each square, that depends of the material type, to encourage optimal placement (e.g., knights in the center, pawns advancing, ).
3. **Endgame adaptation:** Endgame phase is detected when both sides have no queens or every side which has a queen has additionally no other piece or one minorpiece maximum. During this phase, King activity is prioritized and kin-square table is changed to encourage central positions.

The final evaluation is scaled with tanh, ensuring stable reward between -1 and 1. If the state correspond to a game over, the evaluation is +1 for win, -1 for checkmate and 0 for draw.

## 5. Numerical experiments

In this section, we will detail the main results obtained in the study. To do so, we first consider the training models and the achieved performances.

### 5.1. Achieved ELO ratings

To fully evaluate and compare the different models, we define an environment where the deep agent will play against a Stockfish agent. The Stockfish engine allows us to input directly the ELO rating of the bot against which the agent is playing. The minimal ELO rating supported by the engine is 1320. In the Table 1, we computed the Stockfish engine ELO rating such that the agents manage to win the game against the Stockfish bot. In fact, the action choices defined

| Action          | Model             | Achieved ELO |
|-----------------|-------------------|--------------|
| argmax $\pi$    | BT4-Supervised    | 1950         |
|                 | <b>BT4-Distil</b> | <b>2000</b>  |
|                 | BT4-GRPO          | 1900         |
| $\pi$ -sampling | BT4-Supervised    | 1320         |
|                 | BT4-Distil        | 1320         |
|                 | <b>BT4-GRPO</b>   | <b>1550</b>  |

Table 1. Achieved ELO given the action strategy and the agent

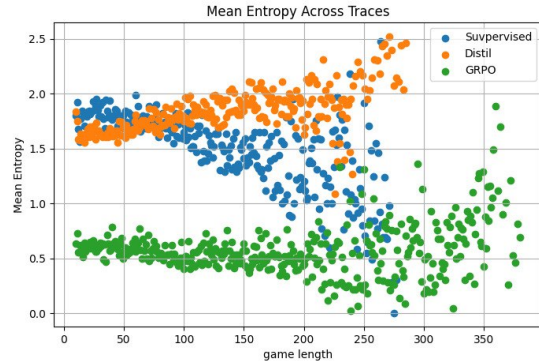


Figure 5. Mean entropy for the different agents

in the table are done using two strategy. The first one is actually a greedy strategy over the policy and the second is a sampling strategy over  $\text{softmax}(\pi)$ .

The best performances are obtained using the greedy strategy especially with the model on which we applied the Distilled learning. Nonetheless, the overall results are really close to each other, so the determination of the best model is not as straightforward as it may seem.

The policy sampling on the other hand, despite lower ELO rating, exhibits interesting results regarding the confidence of the outputted policy. In fact, what we see is that the GRPO-tuned agent achieves overall better ELO ratings than the other models in this setting. An interpretation of this result is that the GRPO-tuned model is actually more confident in the move with highest value in the policy distribution. To corroborate this result, we plotted on Fig. 5 the overall entropy of the sampling distribution defined as  $H(\pi) = -\sum_i \text{softmax}(\pi_i) \log(\text{softmax}(\pi_i))$  averaged throughout several games, in the case of a discrete distribution over the game length which is just the number of past turns at a given state. The results seem more diffused for higher game length but this is due to the fact that we have less games which reach such number of turns. What we can clearly observe is that the GRPO-tuned agent exhibits the lowest entropy value throughout the game length which is a sign of better confidence over the distribution of value

for the policy  $\pi$  considered. The supervised and Distilled agents exhibit similar mean entropy over the game length.

## 5.2. Results and performances of deep learning agents

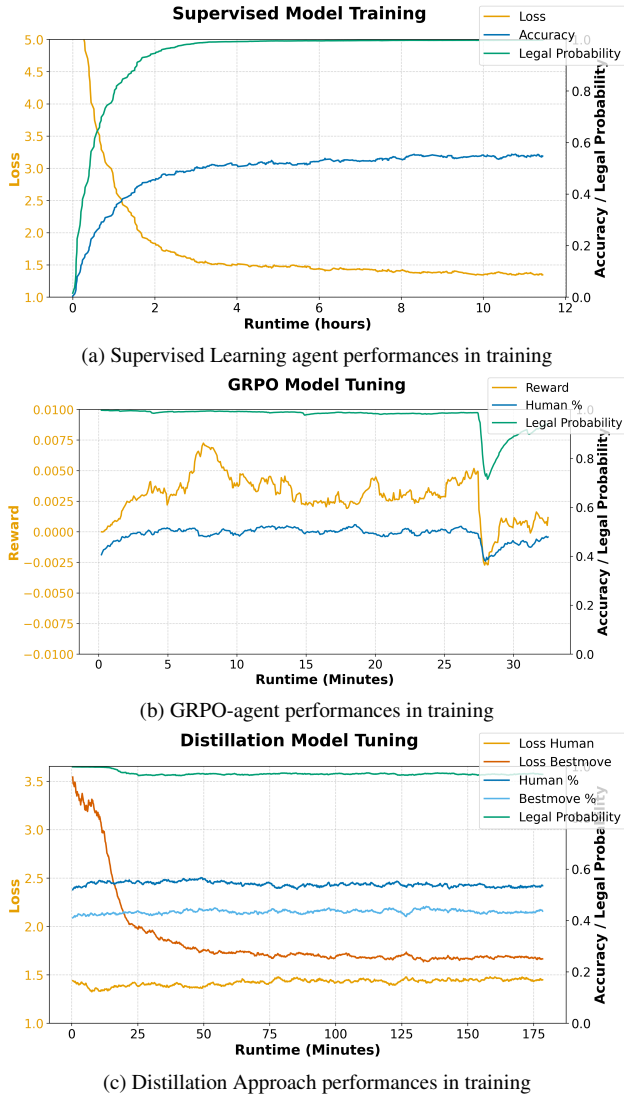


Figure 6. Comparison of different learning approaches: supervised learning, GRPO, and distillation.

In Fig. 6, the different performances of the deep agents are plotted for their training period.

In the case of supervised learning, the model quickly learns to output legal moves, with the legality probability approaching 1, as depicted in Fig. 6a. This indicates that the agent effectively internalizes the rules of chess. The accuracy stabilizes around 55%, meaning the model reproduces human moves approximately 55% of the time, a decent metric for mimicking expert human play.

The GRPO-based agent displays more dynamic behavior. A key metric in Fig. 6b is the surrogate reward, which serves as a proxy for expected reward improvement. Positive values suggest that the agent is outperforming the mean baseline reward, while negative values indicate an under-performance from the model. However, the GRPO method can lead to instability: as the model diverges from the initial reference policy (based on human moves), it may generate increasingly erratic outputs. This can destabilize the training process, leading to fluctuations in loss and potential collapse of the policy. This crash in performance is visible on the Fig. 6b after some time, the model decreases in performances, as the surrogate reward decreases rapidly before coming back up. Adding an additional regularization term to the GRPO loss  $\mathcal{L}_{GRPO}$  to enforce the policy to remain close to a reference policy such as a Stockfish-based or a human-based one may reduce these numerical issues.

Distillation training appears more stable, as detailed in Fig. 6c. The accuracy with respect to human moves (Human %) which measure how similar the agent moves are compared with the human moves and the best moves according to Stockfish (BestMove %) remain relatively steady throughout training. Over time, we may see an improvement of the accuracy with the best move, nonetheless this would require longer training duration. The loss associated with predicting Stockfish’s best move decreases steadily. This indicates that the model is successfully unlearning sub-optimal human moves and aligning more closely with high-quality play recommended by the engine which is the closest to the optimal move at a given board state that we can use.

Overall, each training approach contributes differently to the final agent’s performance: supervised learning provides a strong legal and human-aligned foundation, GRPO pushes the agent toward higher-reward behavior (at the cost of stability), and distillation fine-tunes the agent stably to prefer optimal moves.

## 5.3. Results and performances of classical agents

Overall, our classical chess-playing agents perform significantly worse than the above models (supervised model, GRPO-tuning Distillation-tuning). These advanced models benefit from large-scale training on expert games whereas these tree-based agents start with no prior knowledge and learn purely from in-game search.

Regarding the achieved ELO rating, our agents remain below 1320, which is the lowest bound for Stockfish’s evaluation metrics. Accomplishing more iterations using MCTS allows better performance but requires more computational time. Both agents also benefit from deeper tree-search and simulation but it also requires more computational time.

### Potential Improvements:

- More Sophisticated Evaluation Function: A hand-crafted



evaluation with deeper chess knowledge (pawn structure, king safety, piece activity, captures) could lead to more strategic play. Improving the heuristic function could lead to more efficient pruning using move ordering and thus allow deeper search at same computational cost.

- MCTS rollout policy: Train a neural network to make move selection during the rollout and/or combining this with  $\epsilon$ -greedy policy. However, training a neural network for chess requires large datasets and significant GPU resources, making it impractical for our setup.

## 6. Conclusion & perspectives

While classical methods like MCTS and Alpha-Beta Pruning offer interpretable and effective decision-making through search and heuristics, they remain limited in scalability and performance compared to more recent deep agents. Despite enhancements like rollout policies and handcrafted evaluation functions, their ELO ratings fall below those of pretrained and fine-tuned deep models. In contrast, learning-based approaches—especially those leveraging GRPO or distillation from large-scale training and high-quality supervision, leading to more robust and higher-performing chess agents.

Overall, deep models have achieved impressive performances in regards to a chess environment, not only did they learnt the basic rules of the game highlighted by the legal probability close to 1, but also by their achieved ELO rating. Nonetheless, expansions of the methods presented in this paper can be easily applied to improve the results. Leveraging more computational resources may also improve the achieved ELO.

## References

- [1] Dominik Klein. Neural networks for chess, 2022. [2](#)
- [2] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. [2](#)
- [3] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm, 2017. [1](#), [2](#)
- [4] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018. [1](#)